Module Interface Specification for Crazy 1-0s Game

Team #25, The Crazy Four
Ruida Chen
Ammar Sharbat
Alvin Qian
Jiaming Li

April 5, 2026

1 Revision History

Date	Version	Notes
Nov 12th	Rev-1	Module M1-M11
Nov 12th	Rev-1	Module M12-M22
Nov 13th	Rev-1	Fix consistency
Nov 13th	Rev-1	Fix correlation
Nov 13th	Rev-1	Fix consistency
Jan 14th	Rev0	Remove module 20,21,22
Jan 17th	Rev0	update m1,4,5,6,9,13,16,17,18
Jan 18th	Rev0	update the remaining part
Jan 21th	Rev0	update notation table, fix some errors from feedback
Jan 21th	Rev0	fixed a table format error
Apr 5th	Rev1	Use out:=/exc:= notation, remove void outputs, abstract SQL, add M20, update all modules for NumeralSystem/MathChallenge/new actions

2 Symbols, Abbreviations and Acronyms

See MG Documentation at MG

Contents

1 Revision History **i**

2 Symbols, Abbreviations and Acronyms **ii**

3 Introduction **1**

4 Notation **1**

5 Module Decomposition **2**

6 MIS of API Module (M1) **3**
- 6.1 Module . . . 3
- 6.2 Uses . . . 3
- 6.3 Syntax . . . 3
  - 6.3.1 Exported Constants . . . 3
  - 6.3.2 Exported Access Programs . . . 3
- 6.4 Semantics . . . 4
  - 6.4.1 State Variables . . . 4
  - 6.4.2 Environment Variables . . . 4
  - 6.4.3 Assumptions . . . 4
  - 6.4.4 Access Routine Semantics . . . 4
  - 6.4.5 Local Functions . . . 5
  - 6.4.6 Considerations . . . 6

7 MIS of Real-time Gateway Module (M2) **7**
- 7.1 Module . . . 7
- 7.2 Uses . . . 7
- 7.3 Syntax . . . 7
  - 7.3.1 Exported Constants . . . 7
  - 7.3.2 Exported Access Programs . . . 7
- 7.4 Semantics . . . 8
  - 7.4.1 State Variables . . . 8
  - 7.4.2 Environment Variables . . . 8
  - 7.4.3 Assumptions . . . 8
  - 7.4.4 Access Routine Semantics . . . 8
  - 7.4.5 Local Functions . . . 9
  - 7.4.6 Considerations . . . 9

8 MIS of Matchmaking Module (M3) **10**
- 8.1 Module . . . 10
- 8.2 Uses . . . 10
- 8.3 Syntax . . . 10

8.3.1 Exported Constants . . . 10
8.3.2 Exported Access Programs . . . 10
8.4 Semantics . . . 10
8.4.1 State Variables . . . 10
8.4.2 Environment Variables . . . 11
8.4.3 Assumptions . . . 11
8.4.4 Access Routine Semantics . . . 11
8.4.5 Local Functions . . . 12
8.4.6 Considerations . . . 12

9 MIS of Authentication Module (M4) 13
9.1 Module . . . 13
9.2 Uses . . . 13
9.3 Syntax . . . 13
9.3.1 Exported Constants . . . 13
9.3.2 Exported Access Programs . . . 13
9.4 Semantics . . . 14
9.4.1 State Variables . . . 14
9.4.2 Environment Variables . . . 14
9.4.3 Assumptions . . . 14
9.4.4 Access Routine Semantics . . . 14
9.4.5 Local Functions . . . 16
9.4.6 Considerations . . . 16

10 MIS of Repository Module (M5) 17
10.1 Module . . . 17
10.2 Uses . . . 17
10.3 Syntax . . . 17
10.3.1 Exported Constants . . . 17
10.3.2 Exported Access Programs . . . 18
10.4 Semantics . . . 18
10.4.1 State Variables . . . 18
10.4.2 Environment Variables . . . 18
10.4.3 Assumptions . . . 19
10.4.4 Access Routine Semantics . . . 19
10.4.5 Local Functions . . . 20
10.4.6 Considerations . . . 20

11 MIS of Audit Module (M6) 21
11.1 Module . . . 21
11.2 Uses . . . 21
11.3 Syntax . . . 21
11.3.1 Exported Constants . . . 21

11.3.2 Exported Access Programs . . . 21
11.4 Semantics . . . 21
11.4.1 State Variables . . . 21
11.4.2 Environment Variables . . . 21
11.4.3 Assumptions . . . 22
11.4.4 Access Routine Semantics . . . 22
11.4.5 Local Functions . . . 23
11.4.6 Considerations . . . 23

12 MIS of Real-time Client Module (M7) 24
12.1 Module . . . 24
12.2 Uses . . . 24
12.3 Syntax . . . 24
12.3.1 Exported Constants . . . 24
12.3.2 Exported Access Programs . . . 24
12.4 Semantics . . . 24
12.4.1 State Variables . . . 24
12.4.2 Environment Variables . . . 24
12.4.3 Assumptions . . . 24
12.4.4 Access Routine Semantics . . . 25
12.4.5 Local Functions . . . 25
12.4.6 Considerations . . . 26

13 MIS of Application Shell Module (M8) 27
13.1 Module . . . 27
13.2 Uses . . . 27
13.3 Syntax . . . 27
13.3.1 Exported Constants . . . 27
13.3.2 Exported Access Programs . . . 27
13.4 Semantics . . . 27
13.4.1 State Variables . . . 27
13.4.2 Environment Variables . . . 27
13.4.3 Assumptions . . . 27
13.4.4 Access Routine Semantics . . . 28
13.4.5 Local Functions . . . 28
13.4.6 Considerations . . . 28

14 MIS of Authentication Client Module (M9) 29
14.1 Module . . . 29
14.2 Uses . . . 29
14.3 Syntax . . . 29
14.3.1 Exported Constants . . . 29
14.3.2 Exported Access Programs . . . 29

14.4 Semantics . . . 29
14.4.1 State Variables . . . 29
14.4.2 Environment Variables . . . 29
14.4.3 Assumptions . . . 30
14.4.4 Access Routine Semantics . . . 30
14.4.5 Local Functions . . . 31
14.4.6 Considerations . . . 31

15 MIS of Lobby View Module (M10) 32
15.1 Module . . . 32
15.2 Uses . . . 32
15.3 Syntax . . . 32
15.3.1 Exported Constants . . . 32
15.3.2 Exported Access Programs . . . 32
15.4 Semantics . . . 32
15.4.1 State Variables . . . 32
15.4.2 Environment Variables . . . 33
15.4.3 Assumptions . . . 33
15.4.4 Access Routine Semantics . . . 33
15.4.5 Local Functions . . . 34
15.4.6 Considerations . . . 34

16 MIS of Game Board View Module (M11) 35
16.1 Module . . . 35
16.2 Uses . . . 35
16.3 Syntax . . . 35
16.3.1 Exported Constants . . . 35
16.3.2 Exported Access Programs . . . 35
16.4 Semantics . . . 35
16.4.1 State Variables . . . 35
16.4.2 Environment Variables . . . 35
16.4.3 Assumptions . . . 36
16.4.4 Access Routine Semantics . . . 36
16.4.5 Local Functions . . . 37
16.4.6 Considerations . . . 37

17 MIS of Move Controller Module (M12) 38
17.1 Module . . . 38
17.2 Uses . . . 38
17.3 Syntax . . . 38
17.3.1 Exported Constants . . . 38
17.3.2 Exported Access Programs . . . 38
17.4 Semantics . . . 38

17.4.1 State Variables . . . 38
17.4.2 Environment Variables . . . 38
17.4.3 Assumptions . . . 39
17.4.4 Access Routine Semantics . . . 39
17.4.5 Local Functions . . . 40
17.4.6 Considerations . . . 40

18 MIS of Scoreboard View Module (M13) 41
18.1 Module . . . 41
18.2 Uses . . . 41
18.3 Syntax . . . 41
18.3.1 Exported Constants . . . 41
18.3.2 Exported Access Programs . . . 41
18.4 Semantics . . . 41
18.4.1 State Variables . . . 41
18.4.2 Environment Variables . . . 41
18.4.3 Assumptions . . . 42
18.4.4 Access Routine Semantics . . . 42
18.4.5 Local Functions . . . 43
18.4.6 Considerations . . . 43

19 MIS of Profile View Module (M14) 44
19.1 Module . . . 44
19.2 Uses . . . 44
19.3 Syntax . . . 44
19.3.1 Exported Constants . . . 44
19.3.2 Exported Access Programs . . . 44
19.4 Semantics . . . 44
19.4.1 State Variables . . . 44
19.4.2 Environment Variables . . . 44
19.4.3 Assumptions . . . 45
19.4.4 Access Routine Semantics . . . 45
19.4.5 Local Functions . . . 45
19.4.6 Considerations . . . 46

20 MIS of Game Engine Module (M15) 47
20.1 Module . . . 47
20.2 Uses . . . 47
20.3 Syntax . . . 47
20.3.1 Exported Constants . . . 47
20.3.2 Exported Types . . . 47
20.3.3 Exported Access Programs . . . 47
20.4 Semantics . . . 48

20.4.1 State Variables . . . 48
20.4.2 Environment Variables . . . 48
20.4.3 Assumptions . . . 48
20.4.4 Access Routine Semantics . . . 48
20.4.5 Local Functions . . . 49
20.4.6 Considerations . . . 49

21 MIS of Rules Module (M16) 50
21.1 Module . . . 50
21.2 Uses . . . 50
21.3 Syntax . . . 50
21.3.1 Exported Constants . . . 50
21.3.2 Exported Types . . . 50
21.3.3 Exported Access Programs . . . 50
21.4 Semantics . . . 51
21.4.1 State Variables . . . 51
21.4.2 Environment Variables . . . 51
21.4.3 Assumptions . . . 51
21.4.4 Access Routine Semantics . . . 52
21.4.5 Local Functions . . . 53
21.4.6 Considerations . . . 54

22 MIS of Scoring Module (M17) 55
22.1 Module . . . 55
22.2 Uses . . . 55
22.3 Syntax . . . 55
22.3.1 Exported Constants . . . 55
22.3.2 Exported Access Programs . . . 55
22.4 Semantics . . . 55
22.4.1 State Variables . . . 55
22.4.2 Environment Variables . . . 55
22.4.3 Assumptions . . . 55
22.4.4 Access Routine Semantics . . . 56
22.4.5 Local Functions . . . 56
22.4.6 Considerations . . . 56

23 MIS of Base Conversion Module (M18) 57
23.1 Module . . . 57
23.2 Uses . . . 57
23.3 Syntax . . . 57
23.3.1 Exported Constants . . . 57
23.3.2 Exported Access Programs . . . 57
23.4 Semantics . . . 57

23.4.1 State Variables . . . 57
23.4.2 Environment Variables . . . 57
23.4.3 Assumptions . . . 57
23.4.4 Access Routine Semantics . . . 58
23.4.5 Local Functions . . . 58
23.4.6 Considerations . . . 59

24 MIS of Game Actions Module (M19) 60
24.1 Module . . . 60
24.2 Uses . . . 60
24.3 Syntax . . . 60
24.3.1 Exported Constants . . . 60
24.3.2 Exported Access Programs . . . 60
24.4 Semantics . . . 60
24.4.1 State Variables . . . 60
24.4.2 Environment Variables . . . 60
24.4.3 Assumptions . . . 60
24.4.4 Access Routine Semantics . . . 61
24.4.5 Local Functions . . . 61
24.4.6 Considerations . . . 61

25 MIS of Numeral System Module (M20) 62
25.1 Module . . . 62
25.2 Uses . . . 62
25.3 Syntax . . . 62
25.3.1 Exported Types . . . 62
25.3.2 Exported Constants . . . 62
25.3.3 Exported Access Programs . . . 63
25.4 Semantics . . . 63
25.4.1 State Variables . . . 63
25.4.2 Environment Variables . . . 63
25.4.3 Assumptions . . . 63
25.4.4 Access Routine Semantics . . . 63
25.4.5 Local Functions . . . 63
25.4.6 Considerations . . . 63

26 Appendix 64

3 Introduction

The following document details the Module Interface Specifications for Crazy 1-0s Game

Complementary documents include the System Requirement Specifications and Module Guide. The full documentation and implementation can be found at https://github.com/The-Crazy-Four-Games/Crazy-Eights-Game.

4 Notation

The structure of the MIS for modules comes from **?**, with the addition that template modules have been adapted from **?**. The mathematical notation comes from Chapter 3 of **?**. For instance, the symbol := is used for a multiple assignment statement and conditional rules follow the form $(c_1 \Rightarrow r_1 | c_2 \Rightarrow r_2 | ... | c_n \Rightarrow r_n)$.

The following table summarizes the primitive data types used by Crazy 1-0s Game.

Data Type	**Notation**	**Description**
character	char	a single symbol or digit
integer	$\mathbb{Z}$	a number without a fractional component in (-∞, ∞)
natural number	$\mathbb{N}$	a number without a fractional component in [1, ∞)
real	$\mathbb{R}$	any number in (-∞, ∞)
string	string	an ordered list of characters of any length
List of type T	list(T)	a finite ordered list of elements of type T (order matters)
Set of type T	set(T)	a collection of distinct elements of type T (order does not matter)
Map of type K to V	map(K, V)	a mapping from keys of type K to values of type V
Tuple of types $T_1, T_2, \ldots$	tuple($T_1, T_2, \ldots$)	a finite ordered collection of elements of the specified types
Optional type T	T?	a value of type T or *None*/null (i.e., the value may be absent)

The specification of Crazy 1-0s Game uses some derived data types: sequences, strings, and tuples. Sequences are lists filled with elements of the same data type. Strings are sequences of characters. Tuples contain a list of values, potentially of different types. In addition, Crazy 1-0s Game uses functions, which are defined by the data types of their inputs and outputs. Local functions are described by giving their type signature followed by their specification.

5 Module Decomposition

The following table is taken directly from the Module Guide document for this project.

Level 1	**Level 2**	**Level 3 (Leaf Modules)**
Hardware-Hiding Module		(None)
Behaviour-Hiding Module	Core Domain Logic	M15 M16 M17 M18 M19 M20
	Gameplay Interaction	M3 M10 M11 M12 M13 M14
Software Decision Module	Backend (Server)	M1 M2 M4 M5 M6
	Frontend (Client)	M7 M8 M9

Table 1: Module Hierarchy

6 MIS of API Module (M1)

6.1 Module

API

6.2 Uses

- M4 Authentication Module
- M5 Repository Module
- M6 Audit Module

6.3 Syntax

6.3.1 Exported Constants

- **APIVersion**: string := "v1"

6.3.2 Exported Access Programs

Name	In	Out	Exceptions
postRegister	req: RegisterRequest	HttpResponse	BadRequest, UsernameTaken, WeakPassword
postLogin	req: LoginRequest	HttpResponse	BadRequest, InvalidCredentials, AccountNotFound
postGuestSession	req: GuestSessionRequest	HttpResponse	BadRequest
postLogout	req: LogoutRequest	HttpResponse	BadRequest, InvalidToken
getProfile	req: AuthenticatedRequest	HttpResponse	Unauthorized, RecordNotFound
putProfile	req: UpdateProfileRequest	HttpResponse	BadRequest, Unauthorized, RecordNotFound
postNewGame	req: NewGameRequest	HttpResponse	BadRequest, Unauthorized
getHealth	req: HttpRequest	HttpResponse	None

6.4 Semantics

6.4.1 State Variables

None. (Stateless HTTP routes; no persistent state is maintained by this module.)

6.4.2 Environment Variables

- **HTTPServer**: The web server/runtime that receives HTTP requests and dispatches routes.
- **Auth**: Authentication service provided by M4.
- **Repository**: Persistence interface provided by M5.
- **Audit**: Audit logging service provided by M6.

6.4.3 Assumptions

- Requests and responses follow the REST endpoint structure and payload schemas defined by the system.
- Authentication-protected endpoints include a session token (e.g., `Authorization` header).
- M4, M5, and M6 are correctly configured and available in the server runtime.

6.4.4 Access Routine Semantics

postRegister(*req*)

- transition: *result* := Auth.createAccount(*req*.username, *req*.password); Audit.logAuthEvent("REGIST *req*.username)
- out := `HttpResponse`(*result*.token, *result*.userId)
- exc := (*req* invalid ⇒ `BadRequest` | username exists ⇒ `UsernameTaken` | password weak ⇒ `WeakPassword`)

postLogin(*req*)

- transition: *result* := Auth.login(*req*.username, *req*.password); Audit.logAuthEvent("LOGIN", *req*.username)
- out := `HttpResponse`(*result*.token)
- exc := (*req* invalid ⇒ `BadRequest` | credentials wrong ⇒ `InvalidCredentials` | account missing ⇒ `AccountNotFound`)

postGuestSession(*req*)

- transition: *result* := Auth.createGuestSession(*req*.deviceId); Audit.logAuthEvent("GUEST", *req*.deviceId)
- out := `HttpResponse`(*result*.token)
- exc := (*req* invalid ⇒ `BadRequest`)

postLogout(*req*)

- transition: Auth.logout(*req*.token); Audit.logAuthEvent("LOGOUT", *req*.userId)
- out := `HttpResponse`(success=true)
- exc := (*req* invalid ⇒ `BadRequest` | token invalid ⇒ `InvalidToken`)

getProfile(*req*)

- transition: *claims* := Auth.verifyToken(*req*.token); *profile* := Repository.findPlayer(*claims*.userId); Audit.logSystemEvent("PROFILE_VIEW")
- out := `HttpResponse`(*profile*)
- exc := (token invalid ⇒ `Unauthorized` | player absent ⇒ `RecordNotFound`)

putProfile(*req*)

- transition: *claims* := Auth.verifyToken(*req*.token); *updated* := Repository.updatePlayer(*claims*.userId *req*.data); Audit.logSystemEvent("PROFILE_UPDATE")
- out := `HttpResponse`(*updated*)
- exc := (*req*.data invalid ⇒ `BadRequest` | token invalid ⇒ `Unauthorized` | player absent ⇒ `RecordNotFound`)

postNewGame(*req*)

- transition: *claims* := Auth.verifyToken(*req*.token); *lobby* := Matchmaking.createLobby(*claims*.userId *req*.baseId); Audit.logSystemEvent("GAME_BOOTSTRAP")
- out := `HttpResponse`(*lobby*.lobbyId)
- exc := (*req* invalid ⇒ `BadRequest` | token invalid ⇒ `Unauthorized`)

getHealth(*req*)

- out := `HttpResponse`(status="OK")

6.4.5 Local Functions

None.

6.4.6 Considerations

- **Secret:** REST endpoint structure, request/response payload schemas, and HTTP conventions for stateless backend capabilities.
- **Service:** exposes stateless HTTP routes for authentication, profile management, and bootstrapping new games as defined in the SRS; real-time gameplay is handled separately by M2.
- This module should remain stateless; persistence and security concerns are delegated to M5 and M4, while operational logging is delegated to M6.

7 MIS of Real-time Gateway Module (M2)

7.1 Module

Real-time Gateway

7.2 Uses

- M5 Repository Module
- M6 Audit Module
- M15 Game Engine Module

7.3 Syntax

7.3.1 Exported Constants

None.

7.3.2 Exported Access Programs

Name	In	Out	Exceptions
handleConnection	socket: ClientSocket	–	SessionError
handleJoinGame	socket: ClientSocket, gameId: string	–	SessionError, NotFound
handleSubmitAction	socket: ClientSocket, gameId: string, action: Action	–	InvalidAction, SessionError
registerGameSession	gameId: string, state: GameState	–	SessionError
handleChatSend	socket: ClientSocket, gameId: string, text: string	–	–
handleRequestRestart	socket: ClientSocket, gameId: string	–	SessionError
handleLeaveGame	socket: ClientSocket, gameId: string	–	–

7.4 Semantics

7.4.1 State Variables

- *activeSessions*: map(string, GameState) — the current authoritative game state for each active game session.
- *restartRequests*: map(string, set(string)) — tracks which players have requested restart per game.
- *userSockets*: map(string, string) — maps userId to socketId for targeted events.

7.4.2 Environment Variables

- **RealtimeAdapter**: The bidirectional event-driven runtime managing client connections.
- **ClientSocket**: An opaque reference representing a connected client.

7.4.3 Assumptions

- Client connections are established through the **RealtimeAdapter**.
- Game state transitions are processed exclusively through M15.
- Persistence and logging services (M5, M6) are available.

7.4.4 Access Routine Semantics

handleConnection(*socket*)

- transition: *claims* := Auth.verifyToken(*socket*.token); *userSockets*[*claims*.userId] := *socket*.id; Audit.logAuthEvent("WS_CONNECT", *claims*.userId)
- exc := (token invalid ⇒ `SessionError`)

handleJoinGame(*socket, gameId*)

- transition: *game* := *activeSessions*[*gameId*]; assert *socket*.userId ∈ *game*.players; associate *socket* with room(*gameId*); emit public state and private hand to *socket*
- exc := (*gameId* ∉ *activeSessions* ⇒ `NotFound` | user not in game ⇒ `SessionError`)

handleSubmitAction(*socket, gameId, action*)

- transition: *cur* := *activeSessions*[*gameId*]; *next* := M15.applyAction(*cur*, *action*); *activeSessions*[*gameId*] := *next*; broadcast public state to room; emit private hands to each player; if *next*.status = "GAME_OVER" then persist match results via M5

- exc := (action rejected by M15 ⇒ `InvalidAction` | user not in game ⇒ `SessionError`)

registerGameSession(*gameId, state*)

- transition: $activeSessions[gameId] := state$
- exc := ($gameId \in activeSessions$ ⇒ `SessionError`)

handleChatSend(*socket, gameId, text*)

- transition: broadcast message {from: *socket*.userId, text: *text*[0..200], ts: now()} to room(*gameId*)

handleRequestRestart(*socket, gameId*)

- transition: $restartRequests[gameId] := restartRequests[gameId] \cup \{socket.\text{userId}\}$; if all players $\in restartRequests[gameId]$ then create new game via M15.createGame, $activeSessions[newGameId] := newGame$, notify all players; else notify opponent of pending request
- exc := (game not over ⇒ `SessionError`)

handleLeaveGame(*socket, gameId*)

- transition: remove *socket* from room(*gameId*); $restartRequests := restartRequests \setminus \{gameId\}$; if game ongoing then delete $activeSessions[gameId]$; notify opponent

7.4.5 Local Functions

None.

7.4.6 Considerations

- **Secret:** the mapping between client sockets and authoritative game sessions; restart negotiation protocol; chat relay.
- This module is server-authoritative but delegates all rule enforcement and state transitions to M15.
- Networking concerns are isolated here to avoid coupling game logic with transport mechanisms.

8 MIS of Matchmaking Module (M3)

8.1 Module

Matchmaking

8.2 Uses

- M15 Game Engine Module
- M6 Audit Module

8.3 Syntax

8.3.1 Exported Constants

None.

8.3.2 Exported Access Programs

Name	In	Out	Exceptions
createLobby	hostId: string, hostUsername: string, baseId: BaseId	Lobby	LobbyError
joinLobby	lobbyId: string, userId: string	Lobby	LobbyFull, LobbyNotFound, LobbyAlreadyStarted
startMatch	lobbyId: string, hostId: string	GameId, Lobby, GameState	LobbyNotFound, NotLobbyHost, LobbyError
listOpenRooms	–	list(Lobby)	–
leaveLobby	lobbyId: string, userId: string	–	–
deleteLobby	lobbyId: string	–	–

8.4 Semantics

8.4.1 State Variables

- *lobbies*: map(string, Lobby) — all active waiting lobbies, keyed by lobbyId.

8.4.2 Environment Variables

None.

8.4.3 Assumptions

- User identities are authenticated before invoking this module.
- M2 is available to register real-time game sessions.

8.4.4 Access Routine Semantics

createLobby(*hostId, hostUsername, baseId*)

- transition: *id* := generateUniqueId(); *lobbies*[*id*] := Lobby(*hostId*, *hostUsername*, *baseId*, status="OPEN")
- out := *lobbies*[*id*]
- exc := (creation fails ⇒ `LobbyError`)

joinLobby(*lobbyId, userId*)

- transition: *l* := *lobbies*[*lobbyId*]; *l*.guestId := *userId*
- out := *l*
- exc := (*lobbyId* ∉ *lobbies* ⇒ `LobbyNotFound` | *l*.status ≠ "OPEN" ⇒ `LobbyAlreadyStarted` | *l*.guestId ≠ None ⇒ `LobbyFull`)

startMatch(*lobbyId, hostId*)

- transition: *l* := *lobbies*[*lobbyId*]; *game* := M15.createGame([*l*.hostId, *l*.guestId], *l*.baseId); *l*.status := "STARTED"
- out := (*game*.gameId, *l*, *game*)
- exc := (*lobbyId* ∉ *lobbies* ⇒ `LobbyNotFound` | *hostId* ≠ *l*.hostId ⇒ `NotLobbyHost` | *l*.guestId = None ⇒ `LobbyError`)

listOpenRooms()

- out := {*l* ∈ *lobbies* | *l*.status = "OPEN"}

leaveLobby(*lobbyId, userId*)

- transition: (*userId* = *lobbies*[*lobbyId*].hostId ⇒ delete *lobbies*[*lobbyId*] | *userId* = *lobbies*[*lobbyId*].guest *lobbies*[*lobbyId*].guestId := None)

deleteLobby(*lobbyId*)

- transition: delete *lobbies*[*lobbyId*]

8.4.5 Local Functions

None.

8.4.6 Considerations

- **Secret:** lobby lifecycle management, numeral-system selection, and session handoff logic.
- This module deliberately avoids any game rule or state logic.
- All real-time behavior is delegated to M2 after match creation.

9 MIS of Authentication Module (M4)

9.1 Module

Authentication

9.2 Uses

None.

9.3 Syntax

9.3.1 Exported Constants

None.

9.3.2 Exported Access Programs

Name	In	Out	Exceptions
createAccount	username: string, password: string	AuthResult	UsernameTaken, WeakPassword, CredentialStoreError
login	username: string, password: string	AuthResult	InvalidCredentials, AccountNotFound, CredentialStoreError
createGuestSession	deviceId: string	AuthResult	CredentialStoreError
logout	token: SessionToken	–	InvalidToken
issueToken	userId: UserID, role: UserRole	SessionToken	TokenSigningError
verifyToken	token: SessionToken	TokenClaims	InvalidToken, ExpiredToken, TokenSigningError
refreshToken	token: SessionToken	SessionToken	InvalidToken, ExpiredToken, TokenSigningError

9.4 Semantics

9.4.1 State Variables

- **hashConfig**: Configuration for password hashing (e.g., algorithm selection, cost parameters).
- **tokenSigner**: A component holding token-signing capability (e.g., HMAC secret or private key handle).
- **credentialStore**: Internal credential storage mechanism used by this module (implementation-defined).
- **activeSessions**: A mapping from `SessionToken` to session metadata (e.g., `userId`, `expiry`, `role`). (If sessions are stateless JWTs, this may be minimal or omitted.)

9.4.2 Environment Variables

- **TokenSigningKey**: Secret key material used to sign/verify tokens (stored securely outside the codebase).
- **SystemClock**: Source of current time used for expiry validation.

9.4.3 Assumptions

- Password hashing configuration is correctly set and kept consistent across deployments.
- **TokenSigningKey** is available and protected (not exposed to clients).
- The internal **credentialStore** is available and correctly initialized.

9.4.4 Access Routine Semantics

createAccount(*username, password*)

- transition: Validate *username* format and *password* strength. Hash *password* using **hashConfig**. Store account and credential info in **credentialStore**. Issue a session token for the new account via **issueToken**.
- output: Return `AuthResult` containing (at minimum) a `SessionToken` and user identity information.
- exception: `UsernameTaken` if *username* already exists in **credentialStore**. `WeakPassword` if *password* fails policy. `CredentialStoreError` if storage fails.

login(*username, password*)

- transition: Retrieve credential record for *username* from **credentialStore**. Verify *password* against stored hash. On success, issue a new token via **issueToken**.

- output: Return `AuthResult` containing a valid `SessionToken`.
- exception: `AccountNotFound` if *username* does not exist. `InvalidCredentials` if verification fails. `CredentialStoreError` if retrieval fails.

createGuestSession(*deviceId*)

- transition: Create or reuse a guest identity bound to *deviceId* (policy-defined). Store minimal guest metadata in **credentialStore** if required. Issue a guest token via **issueToken**.
- output: Return `AuthResult` containing a guest `SessionToken`.
- exception: `CredentialStoreError` if storage fails.

logout(*token*)

- transition: Invalidate *token*. If tokens are stateless, record the token (or its identifier) in a denylist until expiry; otherwise remove from **activeSessions**.
- output: `void`.
- exception: `InvalidToken` if *token* is malformed or cannot be invalidated under the configured policy.

issueToken(*userId, role*)

- transition: Construct token claims (e.g., *userId*, *role*, issued-at, expiry). Sign token using **tokenSigner** backed by **TokenSigningKey**.
- output: Return a signed `SessionToken`.
- exception: `TokenSigningError` if signing fails or key material is unavailable.

verifyToken(*token*)

- transition: Verify signature using **TokenSigningKey**. Validate expiry against **SystemClock**. Optionally check denylist / **activeSessions**.
- output: Return `TokenClaims` extracted from the token.
- exception: `InvalidToken` if signature/format validation fails. `ExpiredToken` if expiry has passed. `TokenSigningError` if verification fails due to key issues.

refreshToken(*token*)

- transition: Verify *token* via **verifyToken**. If eligible for refresh, issue a new token with extended expiry via **issueToken**. Optionally revoke the old token (denylist) depending on policy.
- output: Return a new `SessionToken`.
- exception: `InvalidToken` if verification fails. `ExpiredToken` if the token is no longer refreshable. `TokenSigningError` if issuing the new token fails.

9.4.5 Local Functions

None.

9.4.6 Considerations

- **Secret:** password hashing configuration, credential storage details, and token-signing keys.
- **Service:** creates accounts, validates logins, manages guest sessions, and issues/verifies tokens used by the rest of the backend.
- Other backend modules should not handle raw passwords or token signing directly; they should rely on this module's access routines.
- If hashing algorithms or token mechanisms change, modifications are localized to this module; callers remain unchanged.

10 MIS of Repository Module (M5)

10.1 Module

Repository

10.2 Uses

None.

10.3 Syntax

10.3.1 Exported Constants

None.

10.3.2 Exported Access Programs

Name	In	Out	Exceptions
findPlayerByUsername	username: string	Player	RecordNotFound, DatabaseConnectionError
findPlayerById	playerId: PlayerID	Player	RecordNotFound, DatabaseConnectionError
createPlayer	data: PlayerData	Player	UniqueConstraintViolation, DatabaseConnectionError
updatePlayerProfile	playerId: PlayerID, data: PlayerProfileData	Player	RecordNotFound, DatabaseConnectionError
storeCredential	playerId: PlayerID, cred: CredentialData	–	RecordNotFound, UniqueConstraintViolation, DatabaseConnectionError
getCredentialByUsername	username: string	CredentialRecord	RecordNotFound, DatabaseConnectionError
saveMatchResult	result: MatchResult	–	DatabaseConnectionError
getMatchHistory	playerId: PlayerID, limit: nat, offset: nat	seq of MatchResult	DatabaseConnectionError
getPlayerStats	playerId: PlayerID	PlayerStats	RecordNotFound, DatabaseConnectionError
deletePlayer	playerId: PlayerID	–	RecordNotFound, DatabaseConnectionError

10.4 Semantics

10.4.1 State Variables

- *store*: The internal persistent storage driver (implementation-defined).

10.4.2 Environment Variables

- **PersistenceBackend**: The external storage system accessible to this module.

- **ConnectionConfig**: A secure configuration value used to initialize storage connectivity.

10.4.3 Assumptions

- **PersistenceBackend** is running and reachable from the server-side runtime.
- **ConnectionConfig** is provided securely and grants appropriate permissions.
- The data model for players, credentials, match history, and statistics is initialized and consistent with this module's access routines.

10.4.4 Access Routine Semantics

findPlayerByUsername(*username*)

- out := $p \in$ *store*.Players such that p.username = *username*
- exc := ($\nexists$ such $p \Rightarrow$ `RecordNotFound` | store unreachable $\Rightarrow$ `StorageError`)

findPlayerById(*playerId*)

- out := $p \in$ *store*.Players such that p.id = *playerId*
- exc := ($\nexists$ such $p \Rightarrow$ `RecordNotFound` | store unreachable $\Rightarrow$ `StorageError`)

createPlayer(*data*)

- transition: *store*.Players := *store*.Players $\cup$ {new Player from *data*}
- out := the created Player (including generated id)
- exc := (username already exists $\Rightarrow$ `UniqueConstraintViolation` | store unreachable $\Rightarrow$ `StorageError`)

updatePlayerProfile(*playerId, data*)

- transition: p := findPlayerById(*playerId*); update p's fields with *data*
- out := the updated Player
- exc := ($\nexists$ player with *playerId* $\Rightarrow$ `RecordNotFound` | store unreachable $\Rightarrow$ `StorageError`)

storeCredential(*playerId, cred*)

- transition: *store*.Credentials := *store*.Credentials $\cup$ {(*playerId*, *cred*)}
- exc := ($\nexists$ player with *playerId* $\Rightarrow$ `RecordNotFound` | credential already exists $\Rightarrow$ `UniqueConstraintViolation` | store unreachable $\Rightarrow$ `StorageError`)

getCredentialByUsername(*username*)

- out := $c \in$ *store*.Credentials such that c.player.username = *username*
- exc := ($\nexists$ such $c \Rightarrow$ `RecordNotFound` | store unreachable $\Rightarrow$ `StorageError`)

saveMatchResult(*result*)

- transition: *store*.MatchHistory := *store*.MatchHistory $\cup$ {*result*}
- exc := (store unreachable $\Rightarrow$ `StorageError`)

getMatchHistory(*playerId, limit, offset*)

- out := $\langle m_1, \ldots, m_k \rangle$ where each $m_i \in$ *store*.MatchHistory, m_i.playerId = *playerId*, ordered by time descending, $k \leq$ *limit*, starting from *offset*
- exc := (store unreachable $\Rightarrow$ `StorageError`)

getPlayerStats(*playerId*)

- out := aggregated statistics for *playerId* derived from *store*.MatchHistory
- exc := ($\nexists$ player with *playerId* $\Rightarrow$ `RecordNotFound` | store unreachable $\Rightarrow$ `StorageError`)

deletePlayer(*playerId*)

- transition: *store*.Players := *store*.Players $\setminus$ {p | p.id = *playerId*}; cascade removal of associated credentials and match records
- exc := ($\nexists$ player with *playerId* $\Rightarrow$ `RecordNotFound` | store unreachable $\Rightarrow$ `StorageError`)

10.4.5 Local Functions

None.

10.4.6 Considerations

- **Secret:** the storage technology, data model design, and data access strategies (e.g., pooling, transactions).
- **Service:** provides a clean persistence interface for storing and retrieving players, credentials, match history, and statistics, shielding callers from storage specifics.
- This module abstracts all CRUD operations and data model details; callers never interact with the storage technology directly.
- If the persistence technology changes, only this module needs to be rewritten; callers remain unchanged.

11 MIS of Audit Module (M6)

11.1 Module

Audit

11.2 Uses

None.

11.3 Syntax

11.3.1 Exported Constants

- **DefaultRetentionDays**: nat := 30
- **MaxEventPayloadSize**: nat := 8192 (bytes)

11.3.2 Exported Access Programs

Name	In	Out	Exceptions
logAuthEvent	event: AuthAuditEvent	–	LogStoreError
logGameplayEvent	event: GameplayAuditEvent	–	LogStoreError
logSystemEvent	event: SystemAuditEvent	–	LogStoreError
queryAuditEvents	filter: AuditQueryFilter	seq of AuditEvent	LogStoreError
purgeExpiredEvents	retentionDays: nat	nat	LogStoreError
redactEventPayload	eventId: AuditEventID, fields: seq of string	–	RecordNotFound, LogStoreError

11.4 Semantics

11.4.1 State Variables

- **retentionPolicyDays**: nat := **DefaultRetentionDays**
- **logStore**: Internal storage target/driver for audit events (implementation-defined; may be DB table, file sink, external log service).

11.4.2 Environment Variables

- **SystemClock**: Source of current time for timestamps and retention enforcement.

- **AuditStorageConfig**: Configuration describing storage target(s), retention, and access credentials for the logging backend.

11.4.3 Assumptions

- The event schema (fields for authentication/gameplay/system events) matches the storage format used by **logStore**.
- The storage target(s) configured by **AuditStorageConfig** are reachable during normal operation.
- Retention policies are chosen to satisfy operational debugging, security, and compliance requirements.

11.4.4 Access Routine Semantics

logAuthEvent(*event*)

- transition: Validate *event* payload size $\leq$ **MaxEventPayloadSize**. Attach timestamp from **SystemClock**. Serialize and append/store in **logStore** using the authentication-event schema.
- output: `void`.
- exception: `LogStoreError` if storage/append fails.

logGameplayEvent(*event*)

- transition: Validate payload size and required fields. Attach timestamp. Serialize and append/store in **logStore** using the gameplay-event schema.
- output: `void`.
- exception: `LogStoreError` if storage/append fails.

logSystemEvent(*event*)

- transition: Validate payload size and required fields. Attach timestamp. Serialize and append/store in **logStore** using the system-event schema.
- output: `void`.
- exception: `LogStoreError` if storage/append fails.

queryAuditEvents(*filter*)

- transition: Translate *filter* constraints (time window, event type, user/session identifiers, severity, correlation id, etc.) into **logStore** query operations. Execute query against **logStore**. Apply any post-filters (e.g., pagination) as required.

- output: Return a sequence of `AuditEvent` matching *filter*.
- exception: `LogStoreError` if querying fails.

purgeExpiredEvents(*retentionDays*)

- transition: Compute cutoff time = **SystemClock** − *retentionDays*. Delete, archive, or compact entries older than cutoff according to **AuditStorageConfig** policy.
- output: Return the number of events purged (nat).
- exception: `LogStoreError` if purge/archive fails.

redactEventPayload(*eventId, fields*)

- transition: Locate *eventId* in **logStore**. Replace, mask, or remove specified *fields* in the persisted payload (schema-dependent). Persist the redacted version and mark the record as redacted.
- output: `void`.
- exception: `RecordNotFound` if *eventId* does not exist. `LogStoreError` if update/writeback fails.

11.4.5 Local Functions

None.

11.4.6 Considerations

- **Secret:** the exact event schema, retention policy, and storage targets for operational logs.
- **Service:** captures authentication, gameplay, and system events to support debugging, compliance, security, and user inquiries.
- This module is a library utility; it should not impose dependencies on other modules.
- If storage targets or retention strategies change, updates are localized to this module; event producers remain unchanged as long as the API is stable.

12 MIS of Real-time Client Module (M7)

12.1 Module

Real-time Client

12.2 Uses

- M2 Real-time Gateway Module

12.3 Syntax

12.3.1 Exported Constants

None.

12.3.2 Exported Access Programs

Name	In	Out	Exceptions
connect	serverURL: string	–	ConnectionFailed
disconnect	–	–	–
emit	event: string, payload: Data	–	ConnectionFailed
on	event: string, handler: Callback	–	–

12.4 Semantics

12.4.1 State Variables

- **socket**: ClientSocket — Active real-time connection handle.
- **connected**: boolean — Indicates connection status.

12.4.2 Environment Variables

- **ClientRuntime**: Execution environment providing networking support (e.g., WebSocket).

12.4.3 Assumptions

- M2 is reachable at the provided server URL.
- Network connectivity may be intermittent.

12.4.4 Access Routine Semantics

connect(*serverURL*)

- transition:
  - Establishes a real-time connection to M2.
  - Initializes **socket**.
  - Sets **connected** := true on success.
- output: `void`.
- exception: `ConnectionFailed` if the server is unreachable.

disconnect()

- transition:
  - Terminates the active connection.
  - Sets **connected** := false.
  - Releases **socket**.
- output: `void`.

emit(*event, payload*)

- transition:
  - Serializes *payload*.
  - Sends the event and payload to M2 through **socket**.
- output: `void`.
- exception: `ConnectionFailed` if **connected** is false.

on(*event, handler*)

- transition:
  - Registers *handler* to be invoked when M2 emits *event*.
- output: `void`.

12.4.5 Local Functions

None.

12.4.6 Considerations

- **Secret:** connection lifecycle handling and event dispatch mechanism.
- This module abstracts real-time communication details from higher-level client modules.
- No game logic or UI logic is included in this module.

13 MIS of Application Shell Module (M8)

13.1 Module

Application Shell

13.2 Uses

- M9 Authentication Client Module

13.3 Syntax

13.3.1 Exported Constants

None.

13.3.2 Exported Access Programs

Name	In	Out	Exceptions
Render	route: Route, auth: AuthState	ViewModel	RouteNotFound

13.4 Semantics

13.4.1 State Variables

- **currentUser**: User? — The currently authenticated user, if any.
- **currentRoute**: Route — The currently selected navigation route.

13.4.2 Environment Variables

- **NavigationContext**: An external source of navigation state that provides the current route (e.g., a URL path or navigation stack).

13.4.3 Assumptions

- Authentication state (logged-in vs. guest vs. logged-out) is obtained exclusively through M9.
- Route selection and access gating are based on **currentUser** and **currentRoute**.

13.4.4 Access Routine Semantics

Render(*route, auth*)

- transition:
  - Updates **currentRoute** ← *route*.
  - Updates **currentUser** based on *auth* (obtained via M9).
  - Applies access control rules to determine the effective route: if **currentUser** is absent and **currentRoute** requires authentication, then **currentRoute** is set to the login route.
- output: Returns a *ViewModel* describing the global application layout (e.g., header/footer) and the active page/view corresponding to **currentRoute**.
- exception: `RouteNotFound` if **currentRoute** does not match any entry in the routing table.

13.4.5 Local Functions

None.

13.4.6 Considerations

- **Secret:** the application routing table and the layout composition strategy.
- **Service:** provides the top-level structure for the client application and enforces authentication gating by relying only on M9 for auth state.
- This module contains no gameplay logic; it only coordinates navigation and top-level rendering.

14 MIS of Authentication Client Module (M9)

14.1 Module

Authentication Client

14.2 Uses

- M1 API Module

14.3 Syntax

14.3.1 Exported Constants

- **TokenStorageKey**: string := "authToken"

14.3.2 Exported Access Programs

Name	In	Out	Exceptions
renderLoginForm	None	ViewModel	None
renderSignupForm	None	ViewModel	None
submitLogin	username: string, password: string	AuthViewState	InvalidInput, AuthFailed, NetworkError
submitSignup	username: string, password: string	AuthViewState	InvalidInput, AuthFailed, NetworkError
submitLogout	None	AuthViewState	NetworkError
storeToken	token: string	–	TokenStorageError
loadToken	None	OptionalString	TokenStorageError
clearToken	None	–	TokenStorageError

14.4 Semantics

14.4.1 State Variables

- **currentToken**: OptionalString (cached token currently held by the client)
- **authStatus**: AuthStatus (e.g., `LoggedOut`, `LoggingIn`, `LoggedIn`, `Error`)

14.4.2 Environment Variables

- **ClientRuntime**: Browser/client framework runtime that renders UI and handles events.

- **SecureStorage**: Client-side storage mechanism (e.g., cookie/local storage/session storage), implementation-defined.

14.4.3 Assumptions

- The client calls M1 for authentication-related HTTP requests (login/signup/logout).
- Token storage is best-effort secure according to the chosen client storage mechanism.
- Network calls may fail; the UI reports errors to the user.

14.4.4 Access Routine Semantics

renderLoginForm()

- transition: Construct a login form view model (username/password inputs, submit action, error placeholder).
- output: Return the `ViewModel` for the login form.
- exception: None.

renderSignupForm()

- transition: Construct a signup form view model (username/password inputs, submit action, error placeholder).
- output: Return the `ViewModel` for the signup form.
- exception: None.

submitLogin(*username, password*)

- transition: Validate inputs (non-empty, basic format checks). Call M1 authentication route (e.g., `postLogin`). On success, extract token from the response and call **storeToken**. Update **currentToken** and **authStatus**.
- output: Return `AuthViewState` indicating success/failure and any user-facing message.
- exception: `InvalidInput` if validation fails. `AuthFailed` if server rejects credentials. `NetworkError` if request fails.

submitSignup(*username, password*)

- transition: Validate inputs. Call M1 authentication route (e.g., `postRegister`). On success, store returned token via **storeToken**. Update **currentToken** and **authStatus**.
- output: Return `AuthViewState` indicating success/failure and any user-facing message.

- exception: `InvalidInput` if validation fails. `AuthFailed` if server rejects signup. `NetworkError` if request fails.

submitLogout()

- transition: If **currentToken** exists, call M1 logout route (e.g., `postLogout`) best-effort. Call **clearToken**. Set **currentToken** := None and **authStatus** := `LoggedOut`.
- output: Return `AuthViewState` indicating the user is logged out.
- exception: `NetworkError` if the logout request fails (token clearing still proceeds locally).

storeToken(*token*)

- transition: Persist *token* into **SecureStorage** under **TokenStorageKey**. Set **currentToken** := *token*.
- output: `void`.
- exception: `TokenStorageError` if the storage operation fails.

loadToken()

- transition: Read token from **SecureStorage** under **TokenStorageKey**. If present, set **currentToken** := token; otherwise set **currentToken** := None.
- output: Return **currentToken**.
- exception: `TokenStorageError` if the storage read fails.

clearToken()

- transition: Remove **TokenStorageKey** from **SecureStorage**. Set **currentToken** := None.
- output: `void`.
- exception: `TokenStorageError` if removal fails.

14.4.5 Local Functions

None.

14.4.6 Considerations

- **Secret:** decisions about secure token storage and the flows for refreshing or clearing credentials in the browser.
- **Service:** presents login, signup, and logout experiences while coordinating with **M1** for authentication calls.
- **Type Note:** `OptionalString = string` $\cup$ {`None`}.

15 MIS of Lobby View Module (M10)

15.1 Module

Lobby View

15.2 Uses

- M1 API Module
- M7 Real-time Client Module

15.3 Syntax

15.3.1 Exported Constants

None.

15.3.2 Exported Access Programs

Name	In	Out	Exceptions
Render	props: ViewProps	ViewModel	None
handleCreateLobby	token: SessionToken	LobbyID	CreateLobbyError, NetworkError
handleJoinLobby	token: SessionToken, lobbyID: LobbyID	–	JoinLobbyError, NetworkError
handleStartMatch	token: SessionToken, lobbyID: LobbyID	GameID	StartMatchError, NetworkError
subscribeLobbyUpdates	lobbyID: LobbyID	–	ConnectionFailed

15.4 Semantics

15.4.1 State Variables

- **availableLobbies**: seq of LobbySummary — List of lobbies shown to the user.
- **selectedLobby**: LobbyID $\cup$ {None} — Currently selected lobby.
- **statusMessage**: string — UI feedback text (errors/loading).
- **isLoading**: boolean — True while an operation is in progress.

15.4.2 Environment Variables

- **ClientRuntime**: UI runtime responsible for rendering view models and handling user input events.

15.4.3 Assumptions

- The user is authenticated and a valid `SessionToken` is available when creating/joining/starting a match.
- M1 provides HTTP endpoints for lobby and match operations.
- M7 is able to connect to the real-time server and subscribe to lobby/game updates.

15.4.4 Access Routine Semantics

Render(*props*)

- transition: Updates the displayed lobby list and user interaction controls using current state variables.
- output: Returns a `ViewModel` representing the lobby UI (list of lobbies, create/join/start controls, status messages).
- exception: None.

handleCreateLobby(*token*)

- transition:
  - Sets **isLoading** := true.
  - Calls M1 to request lobby creation using *token*.
  - On success, sets **selectedLobby** to the returned `LobbyID`.
- output: Returns the created `LobbyID`.
- exception: `CreateLobbyError` if the server rejects the request; `NetworkError` if the request fails.

handleJoinLobby(*token, lobbyID*)

- transition:
  - Sets **isLoading** := true.
  - Calls M1 to request joining *lobbyID* using *token*.
  - On success, calls **subscribeLobbyUpdates**(*lobbyID*) to listen for real-time status changes.

- output: `void`.
- exception: `JoinLobbyError` if lobby is full/not found/rejected; `NetworkError` if request fails.

handleStartMatch(*token, lobbyID*)

- transition:
  - Calls M1 to start a match for *lobbyID*.
  - Receives a `GameID`.
  - Uses M7 to subscribe to real-time updates for the created game session (implementation-defined event name).
- output: Returns `GameID`.
- exception: `StartMatchError` if the server rejects start; `NetworkError` if request fails.

subscribeLobbyUpdates(*lobbyID*)

- transition:
  - Ensures M7 is connected (calls `M7.connect` if required).
  - Registers event handlers via `M7.on` to receive lobby status updates and update **availableLobbies**/**statusMessage**.
- output: `void`.
- exception: `ConnectionFailed` if M7 cannot connect.

15.4.5 Local Functions

None.

15.4.6 Considerations

- **Secret:** UI-level flow for lobby creation/joining/starting and how lobby updates are presented.
- This module performs no game logic; it only coordinates HTTP calls (M1) and real-time subscriptions (M7).

16 MIS of Game Board View Module (M11)

16.1 Module

Game Board View

16.2 Uses

- M12 Move Controller Module
- M13 Scoreboard View Module

16.3 Syntax

16.3.1 Exported Constants

None.

16.3.2 Exported Access Programs

Name	In	Out	Exceptions
Render	state: RoundState, playerId: PlayerID, scores: ScoreboardData	ViewModel	None
onSelectCard	card: Card	–	None
onPlaySelected	chosenSuit: Suit?	–	InvalidMoveUI, ConnectionFailed
onDraw	–	–	InvalidMoveUI, ConnectionFailed

16.4 Semantics

16.4.1 State Variables

- **selectedCard**: Card ∪ {None} — The card currently selected by the user.
- **validMoves**: seq of Card — Cached playable cards for highlighting.
- **statusMessage**: string — UI message for user feedback (errors/info).

16.4.2 Environment Variables

- **ClientRuntime**: UI runtime that renders the view model and delivers user input events.

16.4.3 Assumptions

- This module is a client-side view and does not modify authoritative game state.
- The current `RoundState` and player identity are provided by the client application flow.
- M12 is available for validating and emitting moves; M13 is available for formatting/displaying scores.

16.4.4 Access Routine Semantics

Render(*state, playerId, scores*)

- transition:
  - Calls `M12.getClientValidMoves(state, playerId)` to update **validMoves**.
  - Calls `M13.renderScoreboard(scores)` to obtain a scoreboard view component.
  - Constructs a `ViewModel` that displays: player hand, discard pile/top card, draw control, turn/status indicators, and the rendered scoreboard.
  - Highlights cards that are in **validMoves**.
- output: Returns the constructed `ViewModel`.
- exception: None. (Errors are represented via **statusMessage** when possible.)

onSelectCard(*card*)

- transition:
  - Sets **selectedCard** := *card*.
  - Clears or updates **statusMessage**.
- output: `void`.

onPlaySelected(*chosenSuit*)

- transition:
  - Requires **selectedCard** $\neq$ None.
  - Calls `M12.handlePlayCard(state, playerId, selectedCard, chosenSuit)`.
  - On failure, updates **statusMessage** with the error context.
- output: `void`.
- exception: `InvalidMoveUI`, `ConnectionFailed`.

onDraw()

- transition:
  - Calls `M12.handleDrawCard(state, playerId)`.
  - On failure, updates **statusMessage**.
- output: `void`.
- exception: `InvalidMoveUI`, `ConnectionFailed`.

16.4.5 Local Functions

None.

16.4.6 Considerations

- **Secret:** the UI layout for presenting the round state and mapping user interactions to controller calls.
- M11 is a presentation module; all move validation and move submission are delegated to M12.
- Score formatting and display are delegated to M13; M11 does not compute scores.

17 MIS of Move Controller Module (M12)

17.1 Module

Move Controller

17.2 Uses

- M7 Real-time Client Module
- M16 Rules Module

17.3 Syntax

17.3.1 Exported Constants

None.

17.3.2 Exported Access Programs

Routine Name	In	Out	Exceptions
handlePlayCard	state: RoundState, playerId: PlayerID, card: Card, chosenSuit: Suit?	–	InvalidMoveUI, ConnectionFailed
handleDrawCard	state: RoundState, playerId: PlayerID	–	InvalidMoveUI, ConnectionFailed
getClientValidMoves	state: RoundState, playerId: PlayerID	seq of Card	InvalidMoveUI

17.4 Semantics

17.4.1 State Variables

- **uiFeedback**: string — Latest user-facing feedback (error/info).

17.4.2 Environment Variables

- **ClientRuntime**: UI runtime that provides user input events and dispatches them to this controller.

17.4.3 Assumptions

- M7 is available and can emit actions to the server (M2).
- The provided `state` is the latest client-visible `RoundState`.
- M16 defines the authoritative rule checks used for client-side pre-validation.

17.4.4 Access Routine Semantics

getClientValidMoves(*state, playerId*)

- transition:
  - Calls `M16.getPlayableCards(state, playerId)`.
- output: Returns the sequence of playable cards for *playerId*.
- exception: `InvalidMoveUI` if the *state* or *playerId* is invalid for rule evaluation.

handlePlayCard(*state, playerId, card, chosenSuit*)

- transition:
  - Performs client-side pre-validation by checking `M16.isPlayable(state, playerId, card)`.
  - If invalid, sets **uiFeedback** and raises `InvalidMoveUI`.
  - If valid, emits a play action to the server via:

    ```
    M7.emit('submitMove', {action: 'play', card: card, chosenSuit: chosenSuit})
    ```

- output: `void`.
- exception:
  - `InvalidMoveUI` if pre-validation fails.
  - `ConnectionFailed` if M7 is not connected.

handleDrawCard(*state, playerId*)

- transition:
  - Performs client-side pre-validation by checking `M16.mustDraw(state, playerId)`.
  - If invalid, sets **uiFeedback** and raises `InvalidMoveUI`.
  - If valid, emits a draw action to the server via:

    ```
    M7.emit('submitMove', {action: 'draw'})
    ```

- output: `void`.
- exception:
  - `InvalidMoveUI` if drawing is not permitted.
  - `ConnectionFailed` if M7 is not connected.

17.4.5 Local Functions

None.

17.4.6 Considerations

- **Secret:** mapping user inputs (clicks, selections) into standardized move events sent to the server.
- Client-side validation is only for responsiveness; the server remains authoritative.
- This module contains no rendering logic and no game-state mutation; it only validates and emits actions.

18 MIS of Scoreboard View Module (M13)

18.1 Module

Scoreboard View

18.2 Uses

- M18 Base Conversion Module

18.3 Syntax

18.3.1 Exported Constants

- **DefaultScoreBase**: nat := 10
- **AlternateScoreBase**: nat := 12

18.3.2 Exported Access Programs

Name	In	Out	Exceptions
renderScoreboard	data: ScoreboardData	ViewModel	InvalidScoreData
toggleScoreBase	base: nat	–	InvalidBase
formatScore	score: int, base: nat	string	InvalidBase, InvalidScoreData
renderRoundSummary	summary: RoundSummaryData	ViewModel	InvalidScoreData
animateRoundTransition	from: ViewModel, to: ViewModel	–	None

18.4 Semantics

18.4.1 State Variables

- **currentBase**: nat := **DefaultScoreBase**
- **lastRendered**: ViewModel (cached view state for transitions/animations)

18.4.2 Environment Variables

- **ClientRuntime**: The client-side framework/runtime responsible for rendering UI components and handling user interactions.
- **DisplayPreferences**: User/UI preferences (e.g., preferred score base, accessibility settings).

18.4.3 Assumptions

- Input scores in `ScoreboardData` are valid integers produced by the game logic.
- Dozenal formatting uses the digit-symbol mapping provided by **M18**.
- The client runtime supports re-rendering and simple transitions/animations.

18.4.4 Access Routine Semantics

renderScoreboard(*data*)

- transition: Validate *data* (players, ordering, score values). For each displayed score, call **formatScore** to produce both decimal and dozenal strings (or the currently selected base). Produce a `ViewModel` suitable for rendering, and update **lastRendered**.
- output: Return the constructed `ViewModel`.
- exception: `InvalidScoreData` if *data* is missing required fields or contains invalid score values.

toggleScoreBase(*base*)

- transition: If *base* is **DefaultScoreBase** or **AlternateScoreBase**, set **currentBase** := *base* and update **DisplayPreferences** if applicable; otherwise reject.
- output: `void`.
- exception: `InvalidBase` if *base* is not supported.

formatScore(*score, base*)

- transition: Validate *score*. If *base* = 10, convert *score* to decimal string using standard formatting. If *base* = 12, call **M18.decimalToDozenal**(*score*) to obtain the dozenal string.
- output: Return the formatted score string.
- exception: `InvalidBase` if *base* is unsupported. `InvalidScoreData` if *score* is not a valid integer (implementation-defined).

renderRoundSummary(*summary*)

- transition: Validate *summary* fields (round index, deltas, totals). Format all displayed score values using **formatScore**. Produce a summary `ViewModel` for end-of-round display.
- output: Return the summary `ViewModel`.

- exception: `InvalidScoreData` if *summary* is malformed or contains invalid scores.

animateRoundTransition(*from, to*)

- transition: Run client-side UI animation/transition from *from* to *to* (implementation-defined). Update **lastRendered** := *to*.
- output: `void`.
- exception: None.

18.4.5 Local Functions

None.

18.4.6 Considerations

- **Secret:** presentation choices for multi-base score displays and animations for round summaries.
- **Service:** shows standings after each round, presenting both decimal and dozenal scores in a clear, accessible format.
- This is a client-side UI module; it should not contain core scoring logic, only formatting and presentation. Numeric base conversion details are delegated to **M18**.

19 MIS of Profile View Module (M14)

19.1 Module

Profile View

19.2 Uses

- M1 API Module

19.3 Syntax

19.3.1 Exported Constants

None.

19.3.2 Exported Access Programs

Name	In	Out	Exceptions
Render	state: ProfileViewState	ViewModel	None
loadProfile	session: SessionToken	–	NetworkError, Unauthorized
updateProfile	session: SessionToken, data: ProfileUpdateData	–	NetworkError, InvalidInput, Unauthorized

19.4 Semantics

19.4.1 State Variables

- **profileData**: ProfileData? — Cached profile information of the current user, if available.
- **isLoading**: boolean — Indicates whether a profile request is in progress.
- **errorMessage**: string? — Error message to be displayed, if any.

19.4.2 Environment Variables

- **InputEvents**: External user interaction events (e.g., edit actions, save action).

19.4.3 Assumptions

- A valid session token is available when invoking **loadProfile** and **updateProfile**.
- Profile data is accessed exclusively through M1; this module does not access persistent storage directly.

19.4.4 Access Routine Semantics

Render(*state*)

- transition: None.
- output: Returns a *ViewModel* that presents the user's profile information (e.g., username, avatar, statistics summary) and indicates whether editing controls are enabled, based on **isLoading** and **errorMessage**.
- exception: None.

loadProfile(*session*)

- transition: Sets **isLoading** ← `true`. Requests the current user's profile via M1. On success, sets **profileData** to the returned profile and clears **errorMessage**. Sets **isLoading** ← `false`.
- output: `void`.
- exception: `Unauthorized` if *session* is invalid or expired. `NetworkError` if the request to M1 fails.

updateProfile(*session, data*)

- transition: Validates *data*. If valid, submits the update request to M1. On success, updates **profileData** to reflect the returned/updated profile and clears **errorMessage**.
- output: `void`.
- exception: `InvalidInput` if *data* fails validation. `Unauthorized` if *session* is invalid or expired. `NetworkError` if the request to M1 fails.

19.4.5 Local Functions

None.

19.4.6 Considerations

- **Secret:** presentation layout decisions and interaction flow for profile rendering and editing.
- **Service:** provides an interface for viewing and updating profile information by delegating all data access to M1.
- This module contains no business logic and no persistence logic; it is a presentation and interaction layer.

20 MIS of Game Engine Module (M15)

20.1 Module

Game Engine

20.2 Uses

- M16 Rules Module
- M17 Scoring Module
- M19 Game Actions Module
- M20 Numeral System Module

20.3 Syntax

20.3.1 Exported Constants

- **MaxPlayers**: $\mathbb{N} := 2$

20.3.2 Exported Types

- `GameStatus` $\in$ {"ONGOING", "GAME_OVER"}
- `GameState` := record(*gameId*: string, *sys*: NumeralSystem, *round*: RoundState, *scoresDec*: map(PlayerID, $\mathbb{Z}$), *status*: GameStatus, *actionLog*: seq(GameAction), *lastSnapshot*: GameState $\cup$ {None}, *activeChallenge*: MathChallenge $\cup$ {None}, *lastRoundResult*: RoundResult $\cup$ {None})
- `RoundResult` := record(*winner*: string, *loser*: string, *pointsGained*: $\mathbb{Z}$, *scoresDec*: map(string, $\mathbb{Z}$))
- `CreateGameOptions` := record(*players*: [PlayerID, PlayerID], *baseId*: BaseId, *initialHandSize*: $\mathbb{N}$, *rngDeck*: seq(Card) $\cup$ {None}, *gameId*: string $\cup$ {None})

20.3.3 Exported Access Programs

Name	In	Out	Exceptions
createGame	opts: CreateGameOptions	GameState	InvalidSetup
applyAction	game: GameState, action: GameAction	GameState	GameOver, NotYourTurn, IllegalMove
undo	game: GameState	GameState	–
getPublicState	game: GameState	PublicState	–

20.4 Semantics

20.4.1 State Variables

None. (This module is a pure function library; game state is passed in and returned.)

20.4.2 Environment Variables

- **RNG**: Source of randomness used for shuffling and dealing.

20.4.3 Assumptions

- This module is the source of truth for game state.
- The rules for legal moves and round transitions are provided by M16.
- Round scoring at the end of a round is provided by M17.
- A game is created with exactly **MaxPlayers** players and a valid *baseId*.

20.4.4 Access Routine Semantics

createGame(*opts*)

- transition: *sys* := M20.getSystem(*opts*.baseId); *round* := M16.initRound(*sys*, *opts*.players, *opts*.initialHandSize)
- out := GameState(*gameId*: generateId(), *sys*, *round*, *scoresDec*: $\{p \mapsto 0 \mid p \in$ *opts*.players$\}$, *status*: "ONGOING", *actionLog*: $\langle\rangle$)
- exc := (*opts*.players invalid $\Rightarrow$ `InvalidSetup`)

applyAction(*game, action*)

- transition: Let *a* := M19.withTimestamp(*action*). Depending on *a*.type:
  - "DRAW" $\Rightarrow$ *round'* := M16.applyDraw(*game*.sys, *game*.round, *a*.playerId)

- "PLAY" ⇒ *round'* := M16.applyPlay(*game*.sys, *game*.round, *a*.playerId, *a*.card, *a*.chosenSuit, *a*.chosenOperation)
- "PASS" ⇒ *round'* := M16.passTurn(*game*.round)
- "ANSWER_CHALLENGE" ⇒ if *a*.answer = *round*.activeChallenge.answer then add reward to scorer, clear challenge; else challenge stays active
- "CHEAT_WIN" ⇒ empty *a*.playerId's hand (triggers round-over)

Auto-pass: if current player has no playable cards and cannot draw, apply M16.passTurn. Round-end: if M16.isRoundOver(*round'*) then *gainDec* := M17.roundGainDec(loser's hand, *game*.sys); update *scoresDec*; if any score ≥ M16.parseInSystem(*sys*.targetScoreText, *sys*) then *status* := "GAME_OVER" else start new round via M16.initRound.

- out := updated GameState with *round'*, appended *actionLog*, saved *lastSnapshot*
- exc := (*game*.status = "GAME_OVER" ⇒ `GameOver` | wrong turn ⇒ `NotYourTurn` | illegal move ⇒ `IllegalMove`)

undo(*game*)

- out := *game*.lastSnapshot if present, else *game*

getPublicState(*game*)

- out := record of non-secret fields: *gameId*, *baseId*, *status*, *turn*, *topCard*, *forcedSuit*, hand-counts (not cards), *scoresDec*, formatted scores in active base via M16.formatInSystem, target score, face ranks, active challenge (with answer removed), last round result

20.4.5 Local Functions

None.

20.4.6 Considerations

- **Secret:** orchestration of round lifecycle, turn sequencing, challenge resolution, auto-pass logic, and how cumulative score/state is stored.
- **Service:** maintains the authoritative state machine for gameplay while delegating legality checks to M16, scoring to M17, and numeral system configuration to M20.
- This module is deterministic given the same shuffled deck seed and action sequence to support testing and debugging.

21 MIS of Rules Module (M16)

21.1 Module

Rules

21.2 Uses

- M18 Base Conversion Module
- M20 Numeral System Module

21.3 Syntax

21.3.1 Exported Constants

- **MaxDrawsPerTurn**: $\mathbb{N} := 3$
- **DefaultHandSize**: $\mathbb{N} := 7$

21.3.2 Exported Types

- `Card` := record(*suit*: Suit, *rank*: string)
- `RoundState` := record(*deck*: seq(Card), *discard*: seq(Card), *topCard*: Card, *forcedSuit*: Suit $\cup$ {None}, *hands*: map(PlayerID, seq(Card)), *players*: [PlayerID, PlayerID], *turn*: PlayerID, *drawCountThisTurn*: $\mathbb{N}$, *freePlayFor*: PlayerID $\cup$ {None}, *activeChallenge*: MathChallenge $\cup$ {None})
- `MathChallenge` := record(*playerId*: string, *type*: $\{+, -, \times, \div\}$, *op1*: $\mathbb{Z}$, *op2*: $\mathbb{Z}$, *answer*: $\mathbb{Z}$, *reward*: $\mathbb{Z}$, *shouldPassTurn*: $\mathbb{B}$)

21.3.3 Exported Access Programs

Name	In	Out	Exceptions
initRound	sys: NumeralSystem, players: [PlayerID,PlayerID], handSize: $\mathbb{N}$	RoundState	InvalidPlayers, DeckTooSmall
isPlayable	sys: NumeralSystem, state: RoundState, playerId: PlayerID, card: Card	$\mathbb{B}$	–
getPlayableCards	sys: NumeralSystem, state: RoundState, playerId: PlayerID	seq(Card)	–
applyPlay	sys: NumeralSystem, state: RoundState, playerId: PlayerID, card: Card, chosenSuit: Suit?, chosenOp: Op?	RoundState	IllegalMove
canDraw	state: RoundState	$\mathbb{B}$	–
applyDraw	sys: NumeralSystem, state: RoundState, playerId: PlayerID	RoundState	NotYourTurn, DrawLimitReached, EmptyDeck
passTurn	state: RoundState	RoundState	–
isRoundOver	state: RoundState	$\mathbb{B}$	–
roundWinner	state: RoundState	PlayerID $\cup$ {None}	–
parseInSystem	text: string, sys: NumeralSystem	$\mathbb{Z}$	InvalidNumberFormat
formatInSystem	n: $\mathbb{Z}$, sys: NumeralSystem	string	InvalidNumberFormat

21.4 Semantics

21.4.1 State Variables

None. (This module is stateless; all game situation is carried in `RoundState`.)

21.4.2 Environment Variables

None.

21.4.3 Assumptions

- This module supports 1v1 gameplay (two players).
- `RoundState` carries the full game situation including draw count, free-play grant, and active challenge.

- The *sys* parameter provides all numeral-system-specific values (wildcard symbols, target sum, face ranks, etc.).
- A **wildcard** card (rank = *sys*.wildcardTenSymbol) is always playable and forces suit selection.
- A **skip** card (rank = *sys*.wildcardSkipSymbol) lets the same player continue.
- Face cards (J, Q, K, and C in dozenal) trigger arithmetic challenges.

21.4.4 Access Routine Semantics

initRound(*sys, players, handSize*)

- transition: Create deck from *sys*; shuffle; deal *handSize* cards per player; set *topCard* from deck; *turn* := players[0]; *drawCountThisTurn* := 0; *freePlayFor* := None
- out := initialized RoundState
- exc := (players[0] = players[1] ⇒ `InvalidPlayers` | deck too small ⇒ `DeckTooSmall`)

isPlayable(*sys, state, playerId, card*)

- out := `true` iff: (*state*.turn = *playerId*) ∧ ¬*state*.activeChallenge ∧ one of:
  - *state*.freePlayFor = *playerId* (free play: any card)
  - *card*.rank = *sys*.wildcardTenSymbol (wildcard always playable)
  - *card*.rank = *sys*.wildcardSkipSymbol (skip always playable)
  - *card*.suit = effectiveSuit(*state*) (match suit)
  - *card*.rank = *state*.topCard.rank (match rank)
  - both numeric ∧ numericValue(*card*) + numericValue(*state*.topCard) = parseInSystem(*sys*.targetSumText, *sys*) (sum rule)

getPlayableCards(*sys, state, playerId*)

- out := $\{c \in state.\text{hands}[playerId] \mid \text{isPlayable}(sys, state, playerId, c)\}$

applyPlay(*sys, state, playerId, card, chosenSuit, chosenOp*)

- transition: Remove *card* from hand; push old topCard to discard; *topCard* := *card*. If wildcard: *forcedSuit* := *chosenSuit*. If skip: *freePlayFor* := *playerId* (same player continues). If face card (J/Q/K/C depending on *sys*): generate MathChallenge with random or chosen operation, set *activeChallenge*. Otherwise advance turn.
- out := updated RoundState

- exc := ($\neg$isPlayable $\Rightarrow$ `IllegalMove`)

canDraw(*state*)

- out := *state*.drawCountThisTurn < **MaxDrawsPerTurn**

applyDraw(*sys, state, playerId*)

- transition: Reshuffle discard into deck if needed; pop card from deck into hand; *drawCountThisTurn* += 1. If *drawCountThisTurn* ≥ 3 and still no playable cards: *turn* := opponent, *freePlayFor* := opponent
- out := updated RoundState
- exc := (not player's turn $\Rightarrow$ `NotYourTurn` | *drawCountThisTurn* $\geq 3 \Rightarrow$ `DrawLimitReached` | deck and discard empty $\Rightarrow$ `EmptyDeck`)

passTurn(*state*)

- transition: *turn* := opponent; *drawCountThisTurn* := 0; *freePlayFor* := None; *forcedSuit* := None
- out := updated RoundState

isRoundOver(*state*)

- out := ($\neg$*state*.activeChallenge) $\wedge$ ($\exists\, p \in$ *state*.players : |*state*.hands[p]| = 0)

roundWinner(*state*)

- out := p where |*state*.hands[p]| = 0, or None if no such p

parseInSystem(*text, sys*)

- out := M18.fromBaseToNumber(*text*, *sys*.spec)
- exc := (invalid format $\Rightarrow$ `InvalidNumberFormat`)

formatInSystem(n, *sys*)

- out := M18.toBaseFromNumber(n, *sys*.spec)

21.4.5 Local Functions

- **effectiveSuit**(*state*): Suit := *state*.forcedSuit if present, else *state*.topCard.suit
- **nextPlayer**(*state*): PlayerID := the player in *state*.players who is not *state*.turn

21.4.6 Considerations

- **Secret:** exact move-validation criteria, including how matching ranks, suits, target sums, wildcards, skips, draw limits, free-play grants, and arithmetic challenges are handled for the active numeral system.
- **Service:** confirms whether a proposed play is legal and enumerates valid plays; provides helpers to initialize rounds and apply state transitions.
- This module does **not** compute scoring; scoring is handled by M17.
- Base conversion is delegated to M18; numeral system configuration comes from M20.

22 MIS of Scoring Module (M17)

22.1 Module

Scoring

22.2 Uses

- M16 Rules Module (for `isFace`, `numericValueDec`, `formatInSystem`)

22.3 Syntax

22.3.1 Exported Constants

None. (Face-card point values are provided by the active NumeralSystem.)

22.3.2 Exported Access Programs

Name	In	Out	Exceptions
handPointsDec	hand: seq(Card), sys: NumeralSystem	$\mathbb{Z}$	–
roundGainDec	loserHand: seq(Card), sys: NumeralSystem	$\mathbb{Z}$	–
roundGain	loserHand: seq(Card), sys: NumeralSystem	record(gainDec: $\mathbb{Z}$, gainText: string)	–

22.4 Semantics

22.4.1 State Variables

None.

22.4.2 Environment Variables

None.

22.4.3 Assumptions

- This module is called only when a round ends (i.e., the winner has emptied their hand).
- The round score equals the sum of the remaining cards in the loser's hand.
- Face cards are worth *sys*.facePointsDec (e.g., 10 in decimal, 12 in dozenal).
- Numeric cards are worth their rank's decimal value (obtained via M16.numericValueDec).

22.4.4 Access Routine Semantics

handPointsDec(*hand, sys*)

- out := $\sum_{c \in hand}$ cardValue(c, *sys*) where

$$\text{cardValue}(c, sys) = \begin{cases} sys.\text{facePointsDec} & \text{if M16.isFace}(c.\text{rank}, sys) \\ \text{M16.numericValueDec}(c.\text{rank}, sys) & \text{otherwise} \end{cases}$$

roundGainDec(*loserHand, sys*)

- out := handPointsDec(*loserHand*, *sys*)

roundGain(*loserHand, sys*)

- out := (*gainDec*: roundGainDec(*loserHand*, *sys*), *gainText*: M16.formatInSystem(*gainDec*, *sys*))

22.4.5 Local Functions

None.

22.4.6 Considerations

- **Secret:** the scoring equation that converts remaining cards into round points, parameterized by the active numeral system's face-card point values.
- **Service:** calculates the round score and returns both a decimal total and a formatted string in the active base.

23 MIS of Base Conversion Module (M18)

23.1 Module

Base Conversion

23.2 Uses

None.

23.3 Syntax

23.3.1 Exported Constants

- **DozenalBase**: nat := 12
- **DecimalBase**: nat := 10

23.3.2 Exported Access Programs

Name	In	Out	Exceptions
decimalToDozenal	n: int	string	InvalidNumberFormat
dozenalToDecimal	s: string	int	InvalidNumberFormat
normalizeDozenal	s: string	string	InvalidNumberFormat
isValidDozenal	s: string	boolean	None

23.4 Semantics

23.4.1 State Variables

None.

23.4.2 Environment Variables

None.

23.4.3 Assumptions

- The dozenal representation uses a fixed, consistent mapping between values and symbols for digits (including the two extra digits beyond 0–9).
- Input strings for conversion do not contain whitespace unless explicitly handled by `normalizeDozenal`.

23.4.4 Access Routine Semantics

decimalToDozenal(n)

- transition: Convert integer n from base **DecimalBase** to base **DozenalBase** using repeated division and remainder; map each remainder to the corresponding dozenal digit symbol; produce the resulting string (including a sign if $n < 0$).
- output: Return a dozenal `string` representing n.
- exception: `InvalidNumberFormat` if n is not a valid finite integer (implementation-defined; e.g., NaN or non-integer input in a loosely-typed context).

dozenalToDecimal(s)

- transition: Normalize s (optional). Validate that every character is a recognized dozenal digit symbol (and optional leading sign). Compute the decimal value by positional evaluation in base **DozenalBase**.
- output: Return the decimal `int` value represented by s.
- exception: `InvalidNumberFormat` if s contains invalid digits/symbols or is otherwise malformed.

normalizeDozenal(s)

- transition: Standardize s into canonical dozenal format (e.g., trim whitespace, normalize casing, map alternate symbols to the project-standard digit symbols).
- output: Return the normalized dozenal `string`.
- exception: `InvalidNumberFormat` if s cannot be normalized into a valid dozenal representation.

isValidDozenal(s)

- transition: Check whether s is a syntactically valid dozenal string under the module's digit-symbol mapping.
- output: Return `true` iff s is valid; otherwise return `false`.
- exception: None.

23.4.5 Local Functions

None.

23.4.6 Considerations

- **Secret:** the mapping of digits and symbols used to move between decimal and dozenal numbers.
- **Service:** translates numeric values to and from dozenal form for scoring logic and UI presentation.
- This is a behaviour-hiding utility module: callers rely on the conversion API without knowing the digit-symbol mapping details.

24 MIS of Game Actions Module (M19)

24.1 Module

Game Actions Module

24.2 Uses

None. (This module defines action data types and utility functions only.)

24.3 Syntax

24.3.1 Exported Constants

None.

24.3.2 Exported Access Programs

Routine Name	In	Out	Exceptions
createAction	ActionType, Parameters	Action	InvalidActionType
validateAction	Action, GameState	Boolean	InvalidMoveException
executeAction	Action, GameState	GameState	ActionExecutionError
undoAction	Action, GameState	GameState	None

24.4 Semantics

24.4.1 State Variables

- **pendingActions**: a queue of unexecuted player actions.
- **lastAction**: most recent action for rollback or replay.

24.4.2 Environment Variables

- Backend execution environment.

24.4.3 Assumptions

- Each action follows the command pattern and can be validated independently.
- The game engine (M15) ensures single-threaded execution for action safety.

24.4.4 Access Routine Semantics

createAction(*ActionType, Parameters*)

- output: constructs an action object from parameters (e.g., "play card 8 spade").

validateAction(*Action, GameState*)

- output: checks if the action is allowed under current rules (by calling M16).

executeAction(*Action, GameState*)

- transition: applies changes to game state (by calling M15) and notifies observers.
- output: `GameState`.

undoAction(*Action, GameState*)

- transition: reverses the last applied change for testing or debugging.
- output: `GameState`.

24.4.5 Local Functions

- `serializeAction()`: converts an action into a string or JSON for replay logging.

24.4.6 Considerations

- This module improves maintainability by isolating gameplay logic into self-contained actions, enabling undo/redo and deterministic testing.

25 MIS of Numeral System Module (M20)

25.1 Module

Numeral System

25.2 Uses

- M18 Base Conversion Module

25.3 Syntax

25.3.1 Exported Types

- `BaseId` ∈ {"dec", "doz"}
- `Suit` ∈ {"S", "H", "D", "C"}
- `NumeralSystem` := record(
  id: BaseId,
  name: string,
  spec: BaseSpec,
  deckNumericSymbols: seq(string),
  faceRanks: seq(string),
  valueOf: map(string, $\mathbb{Z}$),
  targetSumText: string,
  targetScoreText: string,
  wildcardTenSymbol: string,
  wildcardSkipSymbol: string,
  facePointsDec: $\mathbb{Z}$
)

25.3.2 Exported Constants

- **DECIMAL_SYSTEM**: NumeralSystem — Decimal (base-10) configuration: numeric symbols 1–10, face ranks {J, Q, K}, targetSumText = "10", targetScoreText = "100", wildcardTenSymbol = "10", wildcardSkipSymbol = "5", facePointsDec = 10.
- **DOZENAL_SYSTEM**: NumeralSystem — Dozenal (base-12) configuration: numeric symbols 1–↑, face ranks {J, Q, K, C}, targetSumText = "10", targetScoreText = "100", wildcardTenSymbol = "10", wildcardSkipSymbol = "6", facePointsDec = 12.

25.3.3 Exported Access Programs

Name	In	Out	Exceptions
getSystem	id: BaseId	NumeralSystem	–

25.4 Semantics

25.4.1 State Variables

None. (This module is a static registry of numeral system definitions.)

25.4.2 Environment Variables

None.

25.4.3 Assumptions

- The set of supported numeral systems is fixed at compile/deploy time.
- Each NumeralSystem provides a complete, self-consistent configuration (deck symbols, face ranks, wildcards, scoring constants) so that M15, M16, and M17 can operate without hardcoded base-specific logic.

25.4.4 Access Routine Semantics

getSystem(*id*)

- out := the NumeralSystem corresponding to *id* (either **DECIMAL_SYSTEM** or **DOZENAL_SYSTEM**)

25.4.5 Local Functions

None.

25.4.6 Considerations

- **Secret:** the configuration parameters for each supported numeral system.
- **Service:** provides a complete numeral system definition so the game engine, rules, and scoring modules can operate in any supported base without changes to those modules.
- To add a new numeral system (e.g., octal, hexadecimal), one only needs to add a new NumeralSystem constant and register it in this module.

26 Appendix

[Extra information if required —SS]

Appendix — Reflection

[Not required for CAS 741 projects —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Problem Analysis and Design.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?
   The team iterated quickly on TA feedback (e.g., removing implementation-specific wording like JSX/Browser APIs and clarifying what belongs in MG vs MIS). We maintained traceability between the module decomposition/DAG and the MIS sections, so it is clear how modules relate and where responsibilities live.

2. What pain points did you experience during this deliverable, and how did you resolve them?
   During this deliverable, we updated the Module Guide dependency structure (the "Uses" relations and DAG graph) and also merged/split responsibilities across several modules. As a result, many MIS sections no longer matched the architecture and had to be rewritten (not just edited), especially the "Uses" lists, state variables, and access-program contracts. Once the Uses relations were revised, the order in which we wrote MIS modules mattered a lot. We had to start from leaf/low-dependency modules first, then move upward, otherwise we would reference types, exceptions, or semantics that were not yet defined. This created rework when an upstream module's interface changed after a dependent module was already drafted.
   Solution : We stabilized the MG first, then rebuilt the MIS in dependency order (leaf modules $\rightarrow$ higher-level modules). We also created a shared checklist for each module (responsibility, Uses, assumptions, exceptions, and notation consistency) and did a consistency pass to ensure every reference matches the latest MG.

3. Which of your design decisions stemmed from speaking to your client(s) or a proxy (e.g. your peers, stakeholders, potential users)? For those that were not, why, and where did they come from?
   Decisions influenced by client (Dr. Rapoport) feedback: Keeping the game flow simple for 1v1 play (clear turn structure) to reduce user confusion and reduce implementation

risk.
Decisions not directly from clients : The choice of a server-authoritative model came from standard distributed-systems practice for fairness and consistency in multiplayer games, and from risk management

4. While creating the design doc, what parts of your other documents (e.g. requirements, hazard analysis, etc), it any, needed to be changed, and why?
   We updated the MG-to-MIS mapping to reflect changes in the MG dependency structure (e.g., "Uses" relationships and leaf-module grouping) so the MIS matched the latest architecture.

5. What are the limitations of your solution? Put another way, given unlimited resources, what could you do to make the project better? (LO_ProbSolutions)
   Limitations: The current design focuses on 1v1 and a core gameplay loop; scalability to many simultaneous rooms and advanced matchmaking is limited. Security is specified at a design level (auth gating, credential usage), but not hardened like a production deployment.
   With unlimited resources: Add comprehensive automated testing. Expand features (spectator mode, replays, richer profile/stats, better matchmaking, accessibility improvements).

6. Give a brief overview of other design solutions you considered. What are the benefits and tradeoffs of those other designs compared with the chosen design? From all the potential options, why did you select the documented design? (LO_Explores)
   Single monolithic module vs modular decomposition:
   Benefits of monolith: faster to prototype initially, fewer interfaces.
   Tradeoffs: difficult to test; tight coupling; violates information hiding; changes ripple everywhere.
   Reason rejected: course expectations and long-term maintainability favor modular design with clear interfaces.